

```
In [61]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from imblearn.over_sampling import SMOTE
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay,
precision_score, recall_score, f1_score, accuracy_score
```

```
In [62]: df = pd.read_csv('nba_naive.csv')
```

```
In [63]: display(df.head())
```

	fg	3p	ft	reb	ast	stl	blk	tov	target_5yrs	total
0	34.7	25.0	69.9	4.1	1.9	0.4	0.4	1.3	0	266.4
1	29.6	23.5	76.5	2.4	3.7	1.1	0.5	1.6	0	252.0
2	42.2	24.4	67.0	2.2	1.0	0.5	0.3	1.0	0	384.8
3	42.6	22.6	68.9	1.9	0.8	0.6	0.1	1.0	1	330.6
4	52.4	0.0	67.4	2.5	0.3	0.3	0.4	0.8	1	216.0

```
In [64]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1340 entries, 0 to 1339
Data columns (total 11 columns):
#   Column          Non-Null Count  Dtype
---  -
0   fg              1340 non-null   float64
1   3p              1340 non-null   float64
2   ft              1340 non-null   float64
3   reb             1340 non-null   float64
4   ast             1340 non-null   float64
5   stl             1340 non-null   float64
6   blk             1340 non-null   float64
7   tov             1340 non-null   float64
8   target_5yrs     1340 non-null   int64
9   total_points    1340 non-null   float64
10  efficiency       1340 non-null   float64
dtypes: float64(10), int64(1)
memory usage: 115.3 KB
```

```
In [65]: df.shape
```

```
Out[65]: (1340, 11)
```

```
In [66]: df.describe()
```

```
Out[66]:
```

	fg	3p	ft	reb	
count	1340.000000	1340.000000	1340.000000	1340.000000	1340.000000
mean	44.169403	19.149627	70.300299	3.034478	1.550505
std	6.137679	16.051861	10.578479	2.057774	1.471110
min	23.800000	0.000000	0.000000	0.300000	0.000000
25%	40.200000	0.000000	64.700000	1.500000	0.600000
50%	44.100000	22.200000	71.250000	2.500000	1.100000
75%	47.900000	32.500000	77.600000	4.000000	2.000000
max	73.700000	100.000000	100.000000	13.900000	10.600000

```
In [67]: y = df['target_5yrs']
X = df.drop('target_5yrs', axis=1)

print(f"Features (X) shape: {X.shape}")
print(f"Target (y) shape: {y.shape}")
```

```
Features (X) shape: (1340, 10)
```

```
Target (y) shape: (1340,)
```

```
In [68]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42, stratify=y)
```

```
In [69]: smote = SMOTE(random_state=42)
X_train_resampled, y_train_resampled = smote.fit_resample(X_train,
y_train)

print("Shape of X_train before SMOTE:", X_train.shape)
print("Shape of y_train before SMOTE:", y_train.shape)
print("Shape of X_train after SMOTE:", X_train_resampled.shape)
print("Shape of y_train after SMOTE:", y_train_resampled.shape)

print("\nClass balance in y_train before SMOTE:")
display(y_train.value_counts(normalize=True) * 100)

print("\nClass balance in y_train after SMOTE:")
display(y_train_resampled.value_counts(normalize=True) * 100)
```

```
Shape of X_train before SMOTE: (1072, 10)
Shape of y_train before SMOTE: (1072,)
Shape of X_train after SMOTE: (1330, 10)
Shape of y_train after SMOTE: (1330,)
```

```
Class balance in y_train before SMOTE:
```

```
target_5yrs
1    62.033582
0    37.966418
Name: proportion, dtype: float64
```

```
Class balance in y_train after SMOTE:
```

```
target_5yrs
1    50.0
0    50.0
Name: proportion, dtype: float64
```

```
In [70]: gnb_smote = GaussianNB()
gnb_smote.fit(X_train_resampled, y_train_resampled)
y_pred_smote = gnb_smote.predict(X_test)
```

```
In [71]: precision_smote = precision_score(y_test, y_pred_smote)
recall_smote = recall_score(y_test, y_pred_smote)

print(f"Precision after SMOTE: {precision_smote:.4f}")
print(f"Recall after SMOTE: {recall_smote:.4f}")
```

```
Precision after SMOTE: 0.8241
Recall after SMOTE: 0.5361
```

* **Precision: 82.41%** - When the model predicts a player will have a long career (`target_5yrs=1`), it is correct 82.41% of the time. This is critical for minimizing "busts" (false positives).

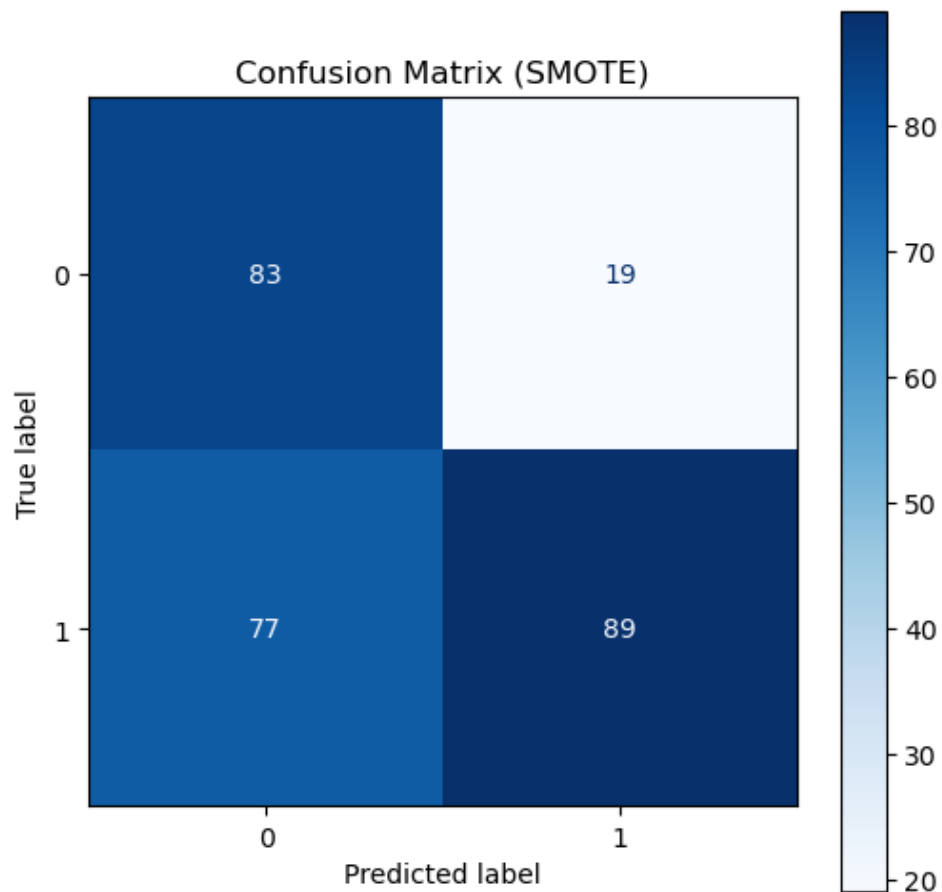
* **Cost of a False Positive (Type I Error) for NBA Scouts:** A false positive means the model predicts a player will have a long career (e.g., 5+ years), but in reality, they do not. For an NBA scout, the cost of a false positive can be significant: it represents **wasted investment**. This could involve spending time, resources, and potentially a draft pick or contract on a player who fails to meet expectations for longevity, leading to financial loss, lost opportunity for other talent, and potential damage to the team's reputation. High precision is generally desirable for scouting to ensure that players identified as having 'long career potential' are genuinely good investments.

* **Recall: 53.61%** - The model identified 53.61% of all players who actually went on to have long careers. This means we are missing about 46% of truly long-career players (false negatives).

* **Cost of a False Negative (Type II Error) for NBA Scouts:** A false negative means the model predicts a player will have a short career (e.g., less than 5 years), but in reality, they go on to have a long and successful career. For an NBA scout, the cost of a false negative is a **missed opportunity**. This means potentially overlooking a "hidden gem" or a future star who could have significantly contributed to the team. While less tangible than the immediate financial cost of a false positive, repeatedly missing out on valuable talent can be detrimental to team building and competitive success in the long run.

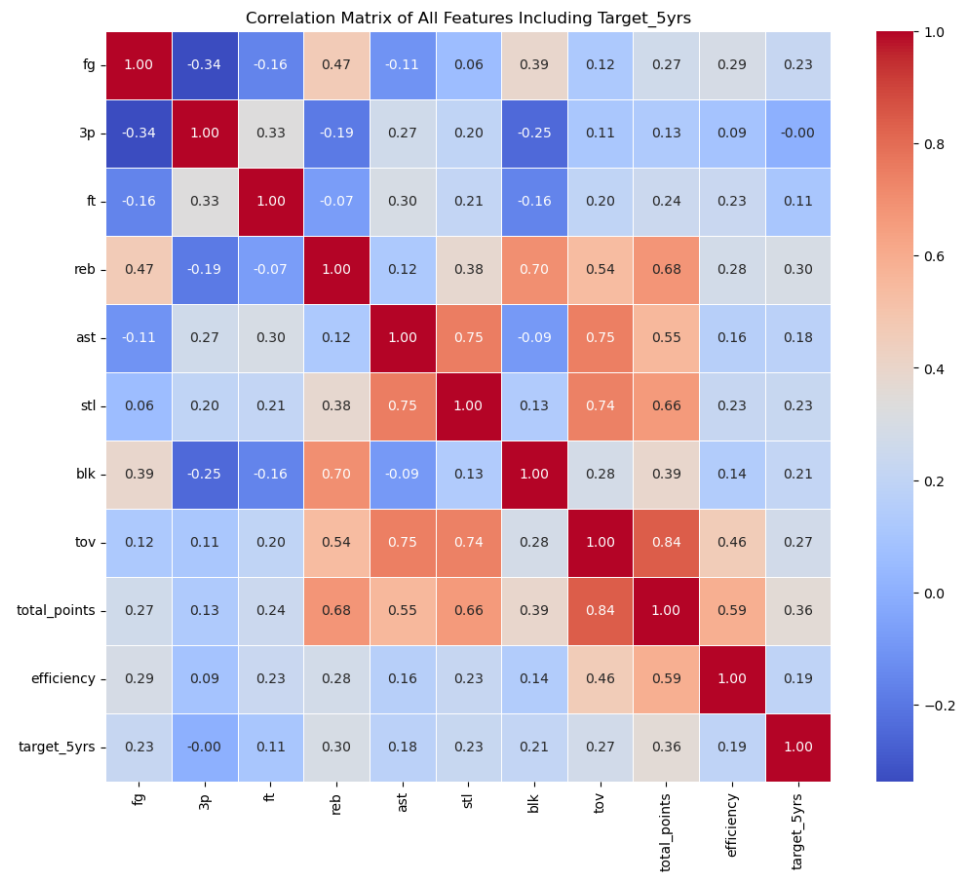
```
In [72]: m_smote = confusion_matrix(y_test, y_pred_smote)
display_smote = ConfusionMatrixDisplay(confusion_matrix=cm_smote,
display_labels=gnb_smote.classes_)

fig, ax = plt.subplots(figsize=(6, 6))
display_smote.plot(ax=ax, cmap='Blues')
plt.title('Confusion Matrix (SMOTE)')
plt.show()
```



```
In [73]: df_full_corr = pd.concat([X, y], axis=1)

plt.figure(figsize=(12, 10))
sns.heatmap(df_full_corr.corr(), annot=True, cmap='coolwarm', fmt='.2f',
            linewidths=.5)
plt.title('Correlation Matrix of All Features Including Target_5yrs')
plt.show()
```



Explanation of the Independence Assumption:

This assumption states that all features (e.g., `fg`, `3p`, `ft`, `reb`, `ast`, etc.) are independent of each other, given the class label (`target_5yrs`). In simpler terms, it assumes that the presence or absence of one feature does not affect the presence or absence of any other feature for a given outcome (whether a player plays 5+ years or not).

Realism for Basketball Stats (e.g., correlation between points and minutes played):

In the context of basketball statistics, the independence assumption is almost certainly **unrealistic**.

Consider the following:

1. **Correlation between points and minutes played:** Players who play more minutes (which would likely be a feature in a more comprehensive dataset) naturally tend to score more points. If 'minutes played' and 'total_points' were both features, they would be highly correlated, violating the independence assumption.
2. **Correlation between assists and turnovers:** Players who handle the ball more to create assists are also more likely to commit turnovers. These features are inherently linked.
3. **Correlation between field goal percentage and total points:** A higher field goal percentage (`fg`) often contributes directly to `total_points`. These are not independent.
4. **Rebounds and blocks:** Certain positions or player types excel in both rebounding and blocking shots, indicating a correlation between these defensive stats.

Because many basketball statistics are interconnected and influence each other (e.g., offensive stats often correlate with each other, and defensive stats with each other, and even offensive with defensive for all-around players), the assumption of independence is largely violated. For example, `total_points` is calculated from other features like `fg`, `3p`, `ft`, meaning it's directly dependent on them.

Impact of the violation:

Despite this theoretical violation, Naive Bayes models can still perform well in practice. This is often because what matters for classification is not necessarily the accurate estimation of probabilities themselves, but rather the ranking of probabilities for different classes. If the independence assumption leads to a consistent (though inaccurate) ranking, the classifier can still make correct predictions.

However, understanding this limitation is crucial. When features are highly correlated, the model might overweight the contribution of these features, as it treats them as separate pieces of evidence. For instance, if 'total_points' and 'fg' are both strong

predictors and highly correlated, the model might count their predictive power twice, which isn't ideal but can still yield good classification performance if the relative weights are maintained.

In summary, while the independence assumption is a pragmatic simplification that makes Naive Bayes computationally efficient, it is not truly reflective of the complex, interdependent nature of basketball statistics.

Most Influential Features

Based on the differences in mean values between players with short vs. long careers, the most influential features driving these longevity predictions are:

1. **total_points (Absolute Mean Difference: 270.62):** This is by far the strongest predictor. Players who play 5+ years accumulate significantly more total points in their initial seasons.
2. **reb (Rebounds, Absolute Mean Difference: 1.27):** Higher rebounding numbers are strongly associated with longer careers.
3. **ast (Assists, Absolute Mean Difference: 0.53):** Players with higher assist totals tend to have greater longevity.
4. **tov (Turnovers, Absolute Mean Difference: 0.41):** Interestingly, players with longer careers tend to have a higher average number of turnovers. This might reflect greater ball-handling responsibilities and playing time.
5. **fg (Field Goal Percentage, Absolute Mean Difference: 2.87):** Better shooting efficiency contributes to career length.

3p (3-point percentage) showed almost no difference between the two groups, suggesting it's not a strong distinguishing factor for longevity in this model.

Actionable Recommendations for Stakeholders

1. **As a First-Pass Filter:** Use this model as a preliminary screening tool to quickly identify players with a high statistical probability of long careers. This can help prioritize scouting efforts.
2. **Combine with Expert Opinion:** Never replace human scouting. Use the model's predictions as one data point among many. If the model flags a player for a short career but scouts are highly enthusiastic, conduct further in-depth analysis.
3. **Investigate Discrepancies:** When the model's prediction strongly contradicts a scout's assessment, use it as an opportunity to delve deeper. Is the player an outlier? Does the model miss a unique skill? Or is the scout's judgment potentially biased?
4. **Monitor Key Metrics:** Pay close attention to `total_points`, `rebounds`, and `assists` in prospective players, as these were identified as the most powerful statistical indicators of longevity by the model.
5. **Consider Model Refinement:** For future iterations, explore more sophisticated models (e.g., Logistic Regression, Random Forests, Gradient Boosting) that do not assume feature independence and can capture more complex relationships. Additionally, incorporate more granular data if available (e.g., minutes played, advanced metrics, college stats, qualitative scouting reports) to enhance predictive power and provide a more holistic view.

In []: